

ACME Project Budget and Resource Tracker

SUMMARY

ACME Inc. is building a centralized project budget and resource tracking application to support its transformation into a more data-driven organization. The MVP will provide a standalone web application that tracks projects, employees, project budgets, employee allocations, actual resource usage, and project health using a RAG status model: Green, Amber, and Red.

BUSINESS PROBLEM

ACME currently lacks a centralized system to answer key project and resource management questions around active projects, project stages, RAG status, allocated and used budget, allocated and used employee hours, employee assignments, overallocation, risk from resource usage outpacing completion, and project dependencies.

MVP PRODUCT VISION

The MVP will be a project portfolio budget and resource tracker that helps ACME monitor project health based on project stage/status, RAG health status, allocated and used project budget, allocated and actual employee/resource hours, employee-to-project assignments, and future dependency and role-based access support. Main value proposition: Project progress + budget burn + employee time usage = project health visibility.

MVP GOALS

1) Create, view, update, and delete projects. 2) Create, view, update, and delete employees. 3) Assign employees to multiple projects. 4) Allocate planned resource hours and cost to projects. 5) Track actual resource usage by project and employee. 6) Track project budget and budget consumption. 7) Calculate and display project RAG status. 8) View project-level summary data. 9) View portfolio-level dashboard metrics. 10) Prepare for future role-based access control. 11) Prepare for future project dependency chain logic.

OUT OF SCOPE FOR MVP

No detailed task management. Out of scope includes Jira-style task boards, tickets, sprints, epics, story points, detailed workflow automation, comments on tasks, file attachments, email notifications, external integrations, employee directory integration, project management tool integration, performance management integration, full role-based access enforcement, and advanced dependency graph visualization.

USER PERSONAS

Admin, Manager, Employee. For MVP, all users may behave as admins and restrictions do not need to be enforced.

ROLE AND ACCESS SCOPE

Future role tiers: Employee, Manager, Admin. MVP: users can register and log in, each user has a role field, default role can be admin, all authenticated users can perform all actions, and backend/database preserve role information for future access control.

CORE MVP DATA CONCEPTS

Employee, Project, Project Budget, Project Resource Allocation, Project Resource Usage, and Project Dependency (stretch goal). Includes many-to-many employee/project assignments via allocations and support for dependency records.

RAG STATUS BUSINESS LOGIC

RAG status is based on actual completion, budget used, and hours used. Calculations: $\text{budget_used_percent} = \text{budget_used} / \text{allocated_budget} * 100$; $\text{hours_used_percent} = \text{hours_used} / \text{allocated_hours} * 100$; $\text{progress_gap} = \max(\text{budget_used_percent}, \text{hours_used_percent}) - \text{actual_completion_percent}$. Rules: Green if $\text{progress_gap} \leq 10$; Amber if $\text{progress_gap} > 10$ and ≤ 25 ; Red if $\text{progress_gap} > 25$. Recalculate on relevant data changes.

MVP FUNCTIONAL REQUIREMENTS

Requirements cover Authentication, Employee Management, Project Management, Project Budget Management, Resource Allocation Management, Resource Usage Management, Project Summary, Dashboard and Reporting, and Project Dependencies.

FRONTEND REQUIREMENTS

Main pages: Login, Dashboard, Projects List, Project Detail, Employees, Employee Detail, Resource Allocation UI, and Resource Usage UI, each with specified required elements.

BACKEND MICROSERVICE REQUIREMENTS

Service domains: Authentication, Employee, Project, Budget, Resource Allocation, Resource Usage, RAG Calculation, Dashboard, and Dependency.

RECOMMENDED API REQUIREMENTS

Authentication: POST /auth/register, POST /auth/login, GET /auth/me. Employees: GET/POST/PUT/DELETE endpoints. Projects: GET/POST/PUT/DELETE and summary endpoint. Budgets, Resource Allocations, Resource Usage, Dependencies, and Dashboard endpoints are defined.

DATABASE REQUIREMENTS

Tables: employees, projects, project_budgets, project_resource_allocations, project_resource_usage, app_users, project_dependencies. Includes many-to-many employees/projects via allocations and related one-to-many/self-referencing relationships.

NON-FUNCTIONAL REQUIREMENTS

Usability, responsiveness, security, performance, maintainability, and serverless readiness. Expected backend stack: Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, JWT, Mangum. Expected frontend stack: React, Material UI.

MVP ACCEPTANCE CRITERIA

Successful MVP includes user auth, employee/project creation, multi-project employee assignment, budget and allocated hours definition, actual usage recording, automatic Green/Amber/Red status, project summary, portfolio dashboard metrics, support for future dependencies and roles, and no Jira-style task management.

FUTURE ENHANCEMENTS

Potential additions include full RBAC enforcement, dependency enforcement and graph visualization, org hierarchy, audit logs, historical snapshots, monthly reporting, CSV export, cloud deployment automation, Terraform, Red-status notifications, advanced forecasting analytics, and integrations.

SUMMARY

The MVP should focus on project budget and resource visibility to help identify healthy projects, over-budget projects, excessive resource-hour consumption, overallocated employees, projects behind budget usage, and potential future dependency blockers, while avoiding becoming a task board.